

Autenticazione e Autorizzazione in OpenShift

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere il flusso di autenticazione in OpenShift: dal comando `oc login` al token Bearer
- Distinguere i principali Identity Providers supportati: HTTPasswd, LDAP, GitHub OAuth, OpenID Connect
- Configurare l'OAuth server tramite la Custom Resource `OAuth` nell'API `config.openshift.io/v1`
- Gestire utenti e identità con `oc get users`, `oc get identity`, `oc whoami`
- Integrare Azure Active Directory / Microsoft Entra ID come OIDC provider su ARO
- Differenziare autenticazione (chi sei) da autorizzazione (cosa puoi fare) in OpenShift

Teoria e Concetti Chiave

Autenticazione vs Autorizzazione

Concetto	Definizione	Componente OpenShift
Autenticazione	Verifica l'identità dell'utente ("chi sei?")	OAuth server, Identity Providers
Autorizzazione	Stabilisce cosa l'utente può fare ("cosa puoi fare?")	RBAC, SCC, API Server

OpenShift separa nettamente i due livelli: l'autenticazione produce un **token Bearer** associato all'utente; l'autorizzazione è poi delegata completamente al sistema RBAC (trattato nel file `02-rbac.md`).

OAuth Server di OpenShift

OpenShift incorpora un **OAuth server** built-in che gestisce il ciclo di vita dei token:

1. Il client (`oc login`, console web, applicazione) invia una richiesta al server OAuth
2. L'OAuth server delega la verifica delle credenziali all'**Identity Provider** configurato
3. Se l'autenticazione ha successo, l'OAuth server emette un **OAuthAccessToken** con scadenza configurabile
4. Il token viene incluso come header `Authorization: Bearer <token>` in ogni richiesta all'API Server

div style="page-break-before: always;" />

Oggetti Kubernetes/OpenShift coinvolti:

Oggetto	API Group	Descrizione
<code>OAuth</code>	<code>config.openshift.io/v1</code>	Configurazione globale dell'OAuth server (cluster-scoped, nome fisso: <code>cluster</code>)
<code>OAuthAccessToken</code>	<code>oauth.openshift.io/v1</code>	Token emesso per una sessione utente

Oggetto	API Group	Descrizione
<code>OAuthClient</code>	<code>oauth.openshift.io/v1</code>	Client registrati che possono richiedere token (console, oc, app custom)
<code>User</code>	<code>user.openshift.io/v1</code>	Rappresentazione dell'utente nel cluster
<code>Identity</code>	<code>user.openshift.io/v1</code>	Mapping tra utente esterno (IDP) e User OpenShift

Identity Providers Supportati

Identity Provider	Tipo	Caso d'uso tipico
<code>HTPasswd</code>	File locale di credenziali	Lab, sviluppo, cluster isolati
<code>LDAP</code>	Integrazione con Active Directory o OpenLDAP	Aziende con AD on-premise
<code>GitHub / GitLab</code>	OAuth 2.0	Team di sviluppo con account GitHub/GitLab
<code>OpenID Connect</code>	OIDC (OAuth 2.0 + JWT)	Azure AD/Entra ID, Google, Okta, Keycloak
<code>Keystone</code>	OpenStack	Ambienti OpenStack
<code>Basic Auth</code>	HTTP Basic + backend remoto	Sistemi legacy
<code>Request Header</code>	Proxy di autenticazione	SSO aziendali con proxy

⚠ Attenzione: Il provider `kubeadmin` (superuser temporaneo) è creato automaticamente all'installazione di OpenShift. Dopo aver configurato un Identity Provider permanente con almeno un utente `cluster-admin`, il provider `kubeadmin` **deve essere rimosso** per ragioni di sicurezza.

Configurazione HTTPasswd (Identity Provider più semplice)

La CR **OAuth** di tipo **HTTPasswd** richiede un Secret che contenga il file httpasswd:

```
# Il campo identityProviders è una lista: si possono configurare più IDP
contemporaneamente
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster          # ← nome fisso, non modificare
spec:
  tokenConfig:
    accessTokenMaxAgeSeconds: 86400 # ← durata massima del token: 24 ore
  identityProviders:
    - name: httpasswd-local          # ← nome logico del provider
      mappingMethod: claim           # ← come mappare l'identità esterna a User
OpenShift
  type: HTTPasswd
  httpasswd:
    fileData:
      name: httpasswd-secret         # ← nome del Secret che contiene il file
httpasswd
```

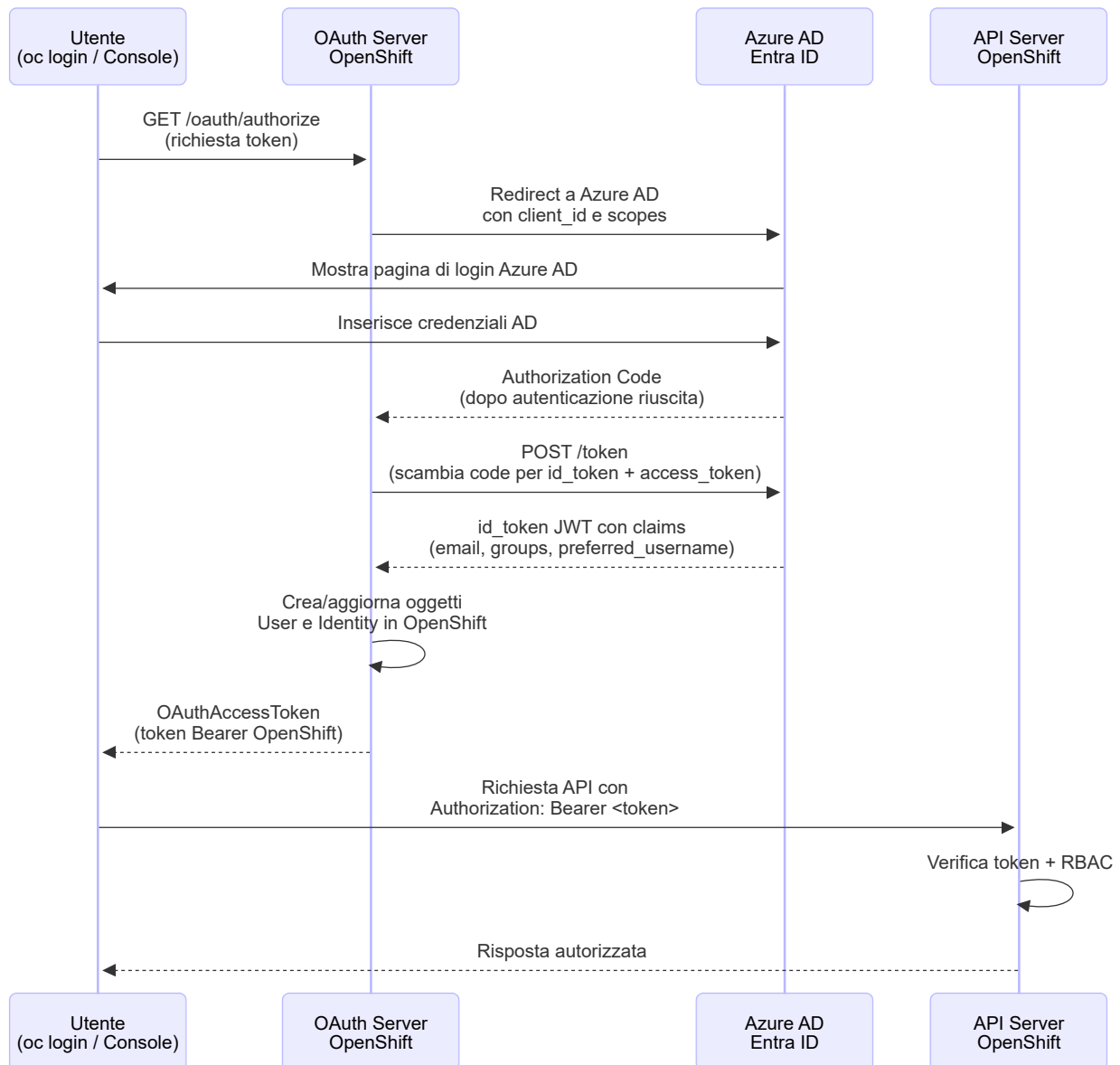
Configurazione OpenID Connect con Azure AD / Entra ID

OpenID Connect è il protocollo usato per integrare Azure Active Directory come Identity Provider su ARO:

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
    - name: azure-ad # ← nome visualizzato nella schermata di
login
    mappingMethod: claim
    type: OpenID
    openID:
      clientID: "<APP_CLIENT_ID>" # ← Application (client) ID della App
Registration in Azure AD
      clientSecret:
        name: azure-ad-client-secret # ← Secret Kubernetes con il valore del
client secret
        issuer: "https://login.microsoftonline.com/<TENANT_ID>/v2.0" # ← Tenant ID
Azure AD
      claims:
        preferredUsername:
          - preferred_username # ← claim JWT usato come username in
OpenShift
          - email
        name:
          - name
        email:
          - email
        groups:
          - groups # ← claim JWT per i gruppi Azure AD
(richiede configurazione app)
      extraScopes:
        - email
        - profile
```

Diagrammi Mermaid

Flusso di Autenticazione OAuth con Azure AD su ARO



Comandi Pratici con Output Atteso

```
# Verificare l'utente correntemente autenticato
oc whoami
```

```
# Output atteso:
# developer@example.com
```

```
# Verificare il token corrente
oc whoami --show-token
```

```
# Output atteso:
# sha256~AbCdEfGhIjKlMnOpQrStUvWxYz1234567890abcdefg
```

```
# Creare il file htpasswd con un nuovo utente
htpasswd -c -B /tmp/htpasswd admin
# (richiede input password)
```

```
# Aggiungere un secondo utente al file esistente
htpasswd -B /tmp/htpasswd developer
```

```
# Creare il Secret con il file htpasswd nel namespace openshift-config
oc create secret generic htpasswd-secret \
  --from-file=htpasswd=/tmp/htpasswd \
  -n openshift-config
```

```
# Output atteso:
# secret/htpasswd-secret created
```

```
# Applicare la configurazione OAuth con HTPasswd
oc apply -f oauth-htpasswd.yaml
```

```
# Output atteso:
# oauth.config.openshift.io/cluster configured
```

```
# Verificare che il deployment dell'OAuth server si sia riavviato
oc rollout status deployment/oauth-openshift -n openshift-authentication
```

```
# Output atteso:
# deployment "oauth-openshift" successfully rolled out
```

```
# Elencare tutti gli utenti registrati nel cluster
oc get users
```

```
# Output atteso:
# NAME                                UID                                FULL NAME
IDENTITIES
# admin                                a1b2c3d4-e5f6-7890-abcd-ef1234567890
htpasswd-local:admin
# developer@example.com               b2c3d4e5-f6a7-8901-bcde-f12345678901      azure-
ad:developer@example.com
```

```
# Elencare le identity (mapping tra IDP e User OpenShift)
oc get identity
```

```
# Output atteso:
# NAME                                IDP NAME                                IDP USER NAME
USER NAME                                USER UID
# htpasswd-local:admin               htpasswd-local                         admin
admin                                a1b2c3d4-...
# azure-ad:developer@example.com     azure-ad                                developer@example.com
developer@example.com               b2c3d4e5-...
```

```
# Assegnare il ruolo cluster-admin a un utente HTPasswd
oc adm policy add-cluster-role-to-user cluster-admin admin
```

```
# Output atteso:
# clusterrole.rbac.authorization.k8s.io/cluster-admin added: "admin"
```

```
# Rimuovere il superuser kubeadmin dopo aver configurato un IDP permanente
oc delete secret kubeadmin -n kube-system
```

```
# Output atteso:
# secret "kubeadmin" deleted
```

Esempi YAML Completi

Secret HTPasswd + Configurazione OAuth Completa

```
# Secret contenente il file htpasswd (generato con `htpasswd -B`)
# Namespace obbligatorio: openshift-config
apiVersion: v1
kind: Secret
metadata:
  name: htpasswd-secret
  namespace: openshift-config # ← namespace obbligatorio per i Secret
  dell'OAuth server
type: Opaque
data:
  # Il valore è il contenuto del file htpasswd codificato in Base64
  # Generare con: htpasswd -nB admin | base64
  htpasswd: YWRtaW46JDJ5JDE0JGtXQ2... # ← base64 del file htpasswd
---
# Configurazione OAuth del cluster con due Identity Providers
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster # ← nome fisso: deve essere "cluster"
spec:
  tokenConfig:
    accessTokenMaxAgeSeconds: 86400 # ← token validi 24 ore
    accessTokenInactivityTimeoutSeconds: 3600 # ← scadenza per inattività: 1 ora
  identityProviders:
    # Provider 1: HTPasswd per utenti locali (admin, emergenza)
    - name: htpasswd-local
      mappingMethod: claim # ← claim = il primo login crea l'utente;
      fail se esiste con altro IDP
      type: HTPasswd
      htpasswd:
        fileName:
          name: htpasswd-secret # ← nome del Secret creato sopra
    # Provider 2: Azure AD per utenti aziendali
    - name: azure-ad
      mappingMethod: claim
      type: OpenID
      openID:
        clientID: "12345678-abcd-ef01-2345-6789abcdef01" # ← App Registration
        client ID
        clientSecret:
          name: azure-ad-client-secret # ← Secret separato con il client secret
        Azure AD
        issuer: "https://login.microsoftonline.com/aaaabbbb-cccc-dddd-eeee-
        ffffgggghhhh/v2.0"
        claims:
          preferredUsername:
            - preferred_username
```



```

      - email
    name:
      - name
    email:
      - email
    groups:
      - groups # ← richiede configurazione "groups claim"
  nell'App Registration Azure AD
  extraScopes:
    - email
    - profile

```

Secret per il Client Secret Azure AD

```

# Secret separato per il client secret dell'App Registration Azure AD
# Namespace obbligatorio: openshift-config
apiVersion: v1
kind: Secret
metadata:
  name: azure-ad-client-secret
  namespace: openshift-config # ← namespace obbligatorio
type: Opaque
stringData:
  clientSecret: "YOUR_AZURE_AD_CLIENT_SECRET_VALUE" # ← valore del client secret
Azure AD

```

Note Specifiche per Azure ARO

Integrazione Azure AD come OIDC Provider

Su ARO, l'integrazione con **Azure Active Directory / Microsoft Entra ID** è il metodo raccomandato per l'autenticazione degli utenti aziendali. Il processo di configurazione richiede:

1. **Creare una App Registration in Azure AD** con le redirect URI dell'OAuth server ARO
2. **Configurare il token di gruppo** nell'App Registration (per ricevere i gruppi come claim nel JWT)
3. **Creare un client secret** nell'App Registration
4. **Applicare la CR OAuth** con tipo **OpenID** puntando al tenant Azure AD

```
# Ottenere l'URL di callback OAuth del cluster ARO
ARO_CLUSTER="my-aro-cluster"
ARO_RG="my-resource-group"

oc get route oauth-openshift -n openshift-authentication \
  -o jsonpath='{.spec.host}/oauth2callback/azure-ad'

# Output atteso:
# https://oauth-openshift.apps.myaro.eastus.aroapp.io/oauth2callback/azure-ad
```

```
# Verificare la configurazione OAuth attuale del cluster ARO
oc get oauth cluster -o yaml

# Creare l'App Registration Azure AD e impostare la redirect URI
az ad app create \
  --display-name "ARO OpenShift Auth" \
  --web-redirect-uris "https://oauth-openshift.apps.myaro.eastus.aroapp.io/oauth2callback/azure-ad"
```

 **Suggerimento:** Su ARO è possibile configurare l'OAuth anche tramite `az aro update --client-id` e `--client-secret` per l'integrazione con la service principal Azure. Tuttavia, per autenticare gli utenti aziendali, la configurazione OIDC tramite App Registration è più flessibile e consigliata.

Mapping Gruppi Azure AD a RoleBinding OpenShift

Dopo aver configurato il claim **groups** nell'App Registration, i gruppi Azure AD sono disponibili come **Group** in OpenShift. È possibile creare RoleBinding che puntano a questi gruppi:

```
# Verificare che i gruppi Azure AD siano stati sincronizzati dopo il login
oc get groups

# Output atteso (dopo che almeno un utente del gruppo si è autenticato):
# NAME                                USERS
# aaaabbbb-cccc-dddd-eeee-ffffgggghhhh developer@example.com, user2@example.com

# Assegnare il ruolo edit al gruppo Azure AD nel namespace myproject
oc adm policy add-role-to-group edit \
  "aaaabbbb-cccc-dddd-eeee-ffffgggghhhh" \
  -n myproject
```

⚠ Attenzione: Su ARO, il cluster ha già un utente **kubeadmin** e una service principal Azure configurata. Non eliminare la service principal ARO: è usata internamente da Azure per gestire il cluster. Eliminare solo il Secret **kubeadmin** dopo aver configurato un IDP permanente con almeno un **cluster-admin**.

Riepilogo dei Punti Chiave

- OpenShift gestisce l'autenticazione tramite un **OAuth server integrato** che delega la verifica delle credenziali agli Identity Providers configurati nella CR **OAuth cluster**
 - I token Bearer emessi dall'OAuth server hanno una durata configurabile (**accessTokenMaxAgeSeconds**) e una scadenza per inattività (**accessTokenInactivityTimeoutSeconds**)
 - I tipi di IDP più comuni sono: **HTPasswd** (laboratori), **LDAP** (AD aziendale), **OpenID** (Azure AD, Google, Okta)
 - Ogni autenticazione riuscita crea o aggiorna gli oggetti **User** e **Identity** nel cluster OpenShift
 - Su ARO, **Azure AD / Microsoft Entra ID** è il provider raccomandato per ambienti aziendali; si configura come **OpenID Connect** con una App Registration Azure AD
 - Il claim **groups** nel JWT Azure AD permette di mappare i **gruppi Azure AD** direttamente a **RoleBinding** OpenShift
 - Dopo la configurazione di un IDP permanente con un utente **cluster-admin**, il Secret **kubeadmin** deve essere **eliminato** per ridurre la superficie di attacco
-

Domande di Verifica

1. Quale oggetto OpenShift contiene la configurazione degli Identity Providers?

- A) `OAuthClient` in `oauth.openshift.io/v1`
- B) `OAuth` in `config.openshift.io/v1` con nome `cluster` ✓
- C) `AuthConfig` in `authentication.openshift.io/v1`
- D) `IdentityProvider` in `user.openshift.io/v1`

2. Un amministratore vuole che gli utenti si autenticino con Azure AD su un cluster ARO. Quale tipo di Identity Provider deve configurare?

- A) `LDAP`
- B) `GitHub`
- C) `OpenID` ✓
- D) `BasicAuth`

3. Cosa rappresenta un oggetto `Identity` in OpenShift?

- A) Un token di accesso emesso dall'OAuth server
- B) Il mapping tra un utente esterno (IDP) e un oggetto `User` OpenShift ✓
- C) Un certificato TLS per l'autenticazione mTLS
- D) Un `ServiceAccount` con permessi speciali

4. Dopo aver configurato `HTPasswd` come Identity Provider e assegnato `cluster-admin` all'utente `admin`, quale azione di sicurezza deve essere eseguita?

- A) Eliminare tutti i `OAuthAccessToken` esistenti
- B) Riconfigurare l'OAuth server con `mappingMethod: lookup`
- C) Eliminare il Secret `kubeadmin` nel namespace `kube-system` ✓
- D) Disabilitare la console web di OpenShift

5. In quale namespace deve essere creato il Secret contenente il file `htpasswd` per l'Identity Provider `HTPasswd`?

- A) `default`
- B) `kube-system`
- C) `openshift-authentication`
- D) `openshift-config` ✓